## Exercice 1 : UTILISATION COMMANDES LINUX & CONFIGURATION THONNY

**1)** Lancer l'application Terminal et (clic souris droit , l'épingler au dashboard pour le conserver).

Dans ce terminal copier et coller la commande pour récupérer le fichier de configuration :

SHELL
```
\$  wget https://www.olivier.baldellon.eu/conf/configuration-web-upinfo.sh
```

taper la commande pour rendre le fichier éxécutable :

SHELL
```
\$  chmod u+x  configuration-web-upinfo.sh
```

Éxécuter ce fichier (pensez à utiliser la touche Tab en cours de saisie)

SHELL
```
\$  ./configuration-web-upinfo.sh
```

Répondre **oui** aux questions qui apparaîssent.

Lancer emacs puis avec la souris cliquer sur **Aide UP info** choisir **installer les paquets** puis attendre la fin du défilement (tout en bas de la fenêtre). Fermer ensuite l'application qui est configurée.

SHELL
```
\$  emacs
```

**2)** Fermer le terminal.

**3)** Ouvrez un nouveau terminal pour prendre en compte les modifications (les couleurs !!). A l'aide des commandes en lignes (maintenant en couleur), déplacez vous dans le répertoire IP2 (avec la commande **cd**) puis créer le répertoire TP06 (avec la commande **mkdir**). Vérifier le résultat de vos actions avec les commandes **ls** et **pwd**.

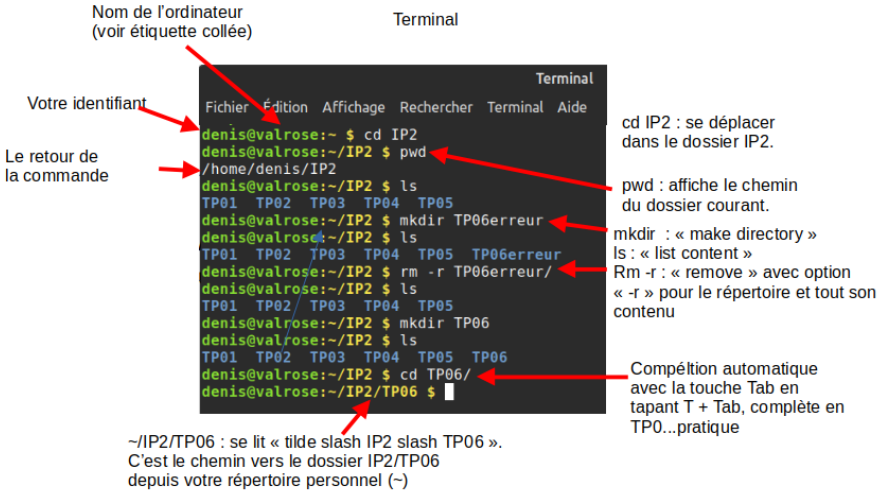


FIGURE 1 – Exemple d'utilisation des commandes pour se déplacer , créer et effacer un répertoire.

Licence Creative Commons

# BIEN RESPECTER LES MAJUSCULES ET MINUSCULES. Si vous changez un nom, vous devez le changer partout y compris dans les fichier python plus tard.

**4)** Dans votre répertoire IP2 , à l'aide des commandes Linux disponibles dans les notes de cours, créer l'arborescence des répertoires de la figure 2 ci-dessous. Cette arborescence doit être placée dans votre répertoire de travail IP2. (cliquer ICI pour accéder au cour Moodle avec les commandes si besoin)

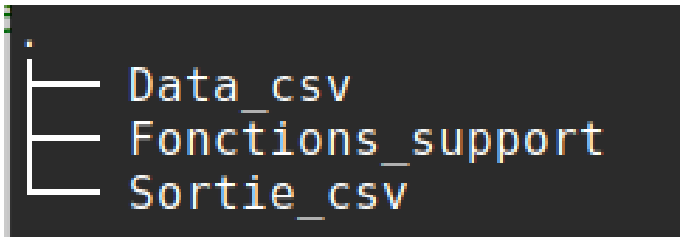


FIGURE 2 – Plan des répertoires.

**5)** Depuis Moodle télécharger les fichiers *Liste_1789_etudiants.csv* et le fichier *gestionFichier.py*.

Ces fichiers après le téléchargement devraient se trouver dans votre répertoire Téléchargement. Vous pouvez consulter le contenu du répertoire (en utilisant la touche Tab pour la complétion) :

SHELL

```
denis@valrose:~/IP2/TP06 $ ls ~/Téléchargements/
gestionFichier.py  Liste_1789_etudiants.csv
denis@valrose:~/IP2/TP06 $
```

**6)** déplacer les fichiers dans les répertoires suivants à l'aide de la commande `mv source destination`

SHELL

```

# La source  (chemin + nom du fichier ) : ~/Téléchargements/gestionFichier.py
# La destination (par exemple , à adapter ) : Fonctions_support

denis@valrose:~/IP2/TP06 $ mv  ~/Téléchargements/gestionFichier.py Fonctions_support
denis@valrose:~/IP2/TP06 $ mv ~/Téléchargements/Liste_1789_etudiants.csv Data_csv
```

Licence Creative Commons

Vérifiez vos déplacements des fichiers :

SHELL

```
denis@valrose:~/IP2/TP06 $ tree
```

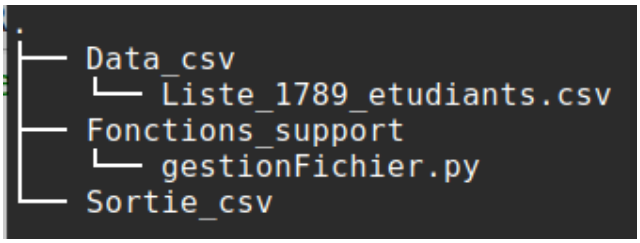


FIGURE 3 – résultat dans le terminal de la commande tree , pour vérifier le bon emplacement de vos fichiers de travail.

**7)** Lancer **Thonny** , dans le menu **Affichage** , vérifier que la ligne **Fichiers** est cochée. Cette option permet d'avoir le navigateur de fichiers visible. Sur la partie gauche de la fenêtre Thonny vous devriez voir votre arborescence avec les fichiers au bon endroit.

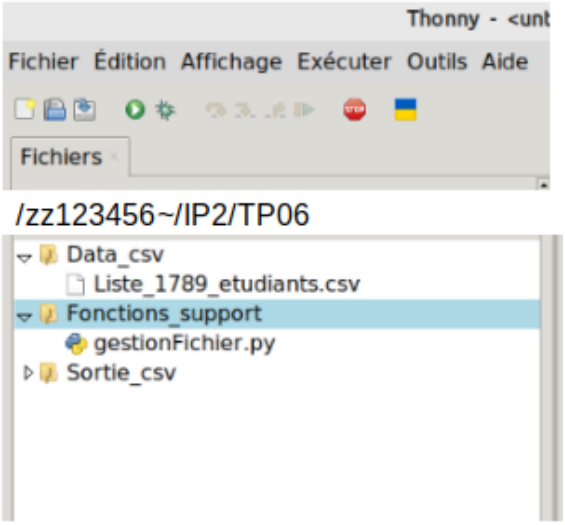


FIGURE 4 – Plan des répertoires avec les fichiers placés.

**8)** Si vous perdez la main sur **Thonny**, vous arrêtez le processus avec pkill. Attention vos fichiers en cours d'utilisation ne seront pas sauvegardés. C'est la raison pour laquelle il faut régulièrement sauvegarder son fichier de travail (CTRL-S)

SHELL

```
denis@valrose:~/IP2/TP06 $ pkill thonny
```

Vous relancez thonny pour la suite.

Licence Creative Commons

**9)** La complétion automatique est très pratique dans le terminal ou dans Thonny. Dans la barre du menu choisir les propositions suivantes : **Outils** puis **Options** et enfin **Éditeur**. Dans la fenêtre, cocher les cases suivantes. En tapant du code Thonny vous fera des complétions possible pour les noms de variable, fonctions qu'il connaît déja.

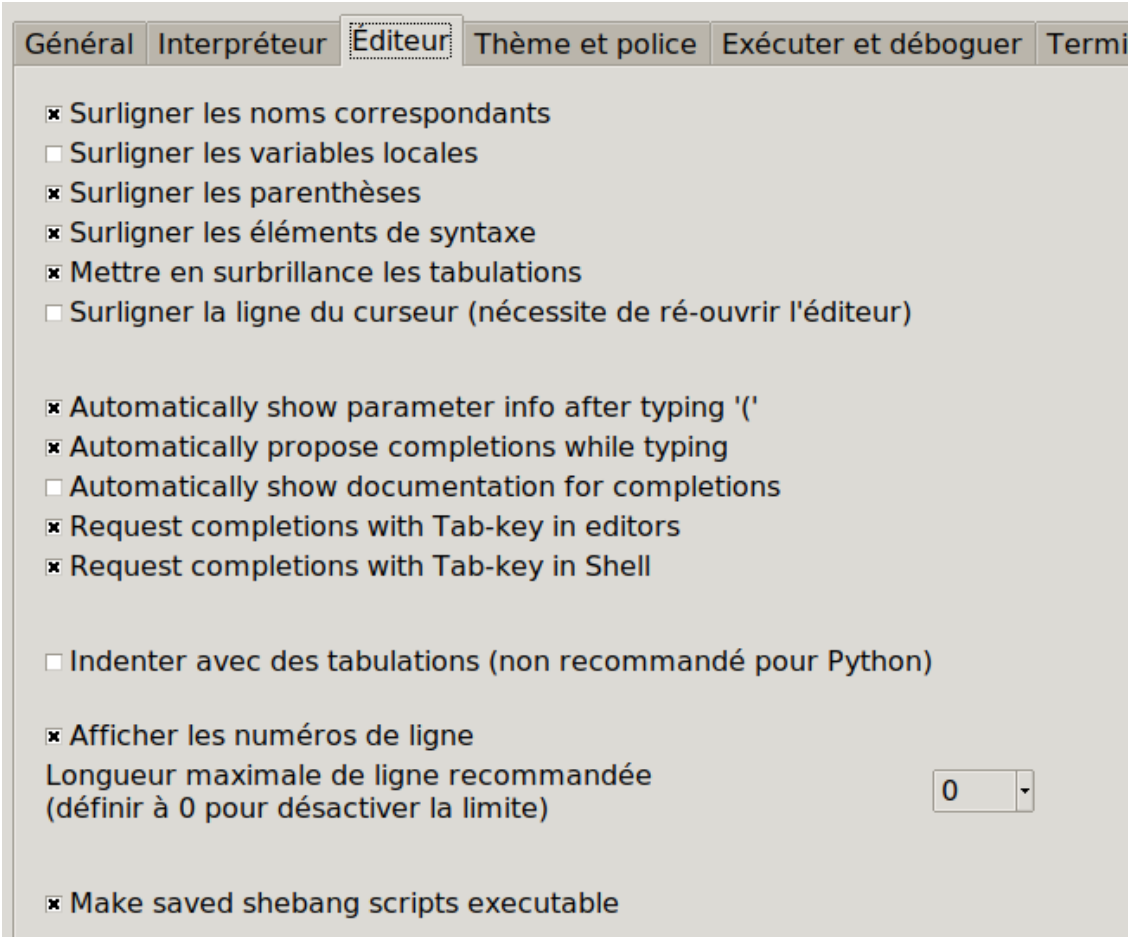


FIGURE 5 – Plan des répertoires avec les fichiers placés.

Licence Creative Commons

## Exercice 2 :

**1) INTRODUCTION :** L'ambassade de France à Bruxelles organise, à l'occasion de la fête nationale du 14 juillet, une réception à laquelle sont conviés 1789 étudiants. Les inscriptions ont été enregistrées par un stagiaire dans un unique fichier, sans "structuration particulière" en langage diplomatique (traduire par "c'est le bazar !").

Le fichier *Liste_1789_etudiants.csv* contient les informations relatives aux invités (nom, prénom et nationalité). Ces identités sont générées aléatoirement, avec le module Faker de Python.

```
Prénom ;Nom de famille ;Pays
NICOLAS ;MILLET ;France
NURI ;HOFFMANN-RADISCH ;Allemagne
ILJA ;ULLMANN-HENCK ;Allemagne
JASPER ;AMALRADA ;Pays-Bas
KRISTINA ;HEINONEN ;Finlande
...
```

Votre travail, détaillé dans les questions 2) et suivantes , consiste en résumé à :

- déterminer le nombre de nationalités représentées et l'effectif d'étudiants associé à chacune d'elles ;
- générer des fichiers au format `csv`, dont le contenu est trié par ordre alphabétique (uniquement suivant les noms, pas sur les prénoms), et les enregistrer dans le répertoire `Sortie_csv` (voir la figure 3 sur la page précédente.)

**2)** Ouvir le fichier *gestionFichier.py* du répertoire *Fonctions_support* dans votre éditeur Python.

Avant de poursuivre, vous devez effectuer les vérifications suivantes :

- Ajouter un bloc d'auto-test à la fin du fichier en utilisant la structure : `if __name__ == "__main__":`
- Tester votre fonction sur le fichier *Liste_1789_etudiants.csv* située dans le répertoire Data_csv.

  **Indication :** depuis le répertoire *Fonctions_support*, le chemin relatif vers *Data_csv* est : *../Data_csv*. Ce chemin doit être utilisé devant le nom du fichier dans la commande de chargement placée dans le bloc d'auto-test.
- Vérifier que le fichier contient bien 1789 lignes de données (sans compter l'entête ).

Licence Creative Commons

test

```
# comme pour le TP 4-5 avec les prénoms, parcourir les données chargées
# et vérifier que chaque ligne est bien composée de 3 chaînes de caractères.
print(data[0])   # pour visualiser un peu les données.
print(data[1])
for ligne in data :
    assert  len(ligne)==3     # 3 si séparateur de données est correct
assert len(data)== ???  # à définir correctement.
```

**3)** Toujours dans le bloc de l'auto test, tester la fonction d'écriture des données dans un fichier CSV.

- Tester la fonction avec le fichier *Liste_1789_etudiants.csv*. Par exemple :
  - charger ce fichier avec votre fonction de lecture,
  - sauvegarder les données dans un nouveau fichier CSV (par exemple *votre_fichier.csv* à placer dans le répertoire *Sortie_csv* en définissant correctement le chemin relatif entre les 2 répertoires.).
- Afficher ensuite les différences des deux fichiers avec la commande *diff* dans le terminal. Placez vous d'abord (avec la commande cd) dans le répertoire TP06 :

SHELL

```
denis@valrose ~/IP2/TP06 $ diff Data_csv/Liste_1789_etudiant.csv
                                                Sortie_csv/votre_fichier.csv
```

Cette commande affiche uniquement les différences entre les deux fichiers. Si aucune différence n'est signalée, cela signifie que le chargement puis l'écriture du fichier est correcte.

**4)** Dans Thonny ou votre éditeur Python.

- Ouvir un fichier vide ou directement CTRL N
- Sauvegarder ce fichier (CTRL S) avec le nom *main.py* dans le répertoire de travail *TP06*.
- Dans le navigateur Thonny, vérifier l'apparition du fichier dans le bon répertoire.
- Dans le terminal taper la commande ls pour voir le nom du fichier.

SHELL

```
denis@valrose ~/IP2/TP06 $ ls
```

Licence Creative Commons

**5)** Dans le fichier *main.py* écrire la fonction qui permute 2 valeurs d'une liste :

permute.py

```
def permute(i : int, j : int , L : list[list[str] ] ) -> None
    """Cette fonction permute les éléments i et j de la liste"""

# test
Liste=[ ['a','b','c'] , ['d','e','f'] ]
print(f"Avant : {Liste}")
permute(i = 0 , j = 1 , L=Liste )
print(f"Apres: {Liste}")
assert Liste[1]==['a','b','c']
```

**6)** Dans le fichier main.py écrire la fonction qui compare l'ordre alphabétique de deux chaînes de caractères. Vous devez utiliser la fonction ord() pour comparer les valeurs ASCII des caractères et aucune méthode de comparaison de chaîne de Python.

est_avant.py

```
def est_avant_bool( nom1 : str , nom2 : str ) -> bool :
    """1Cette fonction compare l'ordre alphabétique entre 2 chaines
    renvoie True si nom1 est avant nom2
    renvoie True si nom1 = nom2"""

# test
assert est_avant_bool('A','B')
assert est_avant_bool('Z','A') == False
assert est_avant_bool('B','B')
assert  est_avant_bool('TOTO','TOTO')
assert  est_avant_bool('TOTO','TOTOTO')
assert  est_avant_bool('TOTOTOTOTO','TO')== False
assert  est_avant_bool('Z','AA')== False
assert  est_avant_bool('ZZ','A')== False
```

Licence Creative Commons

**7)** Dans le fichier *main.py* écrire la fonction qui extrait le nom des pays. Les noms sont renvoyés dans un ensemble. L'intérêt est que les valeurs sont uniques , si le pays "France" est ajouté plusieurs fois, il restera unique, il n'y a pas de doublons dans un ensemble.

extrait

```
def extraitPays(data :  list[list[str]] )->set :
    """extrait l'ensemble des pays"""

E = extraitPays(data)
print(f"Le fichier contient {len(E)} pays")
print(E)
# test minimaliste
assert len(E)==7
```

**8)** Écrire la fonction qui renvoie un dictionnaire dont les clés sont les noms des pays extraits du fichier. Les valeurs du dictionnaire sont initialisées avec des listes vides.

extrait

```
def definitDicoVide( ensembleCle : set) -> dict :

dico = definitDicoVide(E)
assert len(dico)==7
assert len(dico['France'])==0
```

**9)** Écrire la fonction qui parcourt les données et remplit le dictionnaire. Par exemple si le fichier contient la ligne "JEAN","BON","France" alors la liste du dictionnaire correspondant à la clé "France" sera complétée avec la liste ["JEAN","BON"].

La fonction doit :

- lire chaque ligne
- récupérer prénom, nom, pays
- ajouter [prenom, nom] dans la liste correspondant au pays dans le dictionnaire.

```
def remplitDico (dico : dict , data : list[list[str]]) -> None :

remplitDico (dico  , data )
for pays in dico.keys() :   # parcourt les clés du dictionaire
    liste = dico[pays]
    print(f" Il y a {len(liste)} étudiants invités du pays :  {pays}")
```

Licence Creative Commons

**10)** Pour trier les listes de noms des étudiants par pays nous allons ré-utiliser en l'adaptant la fonction de tri dit à bulle du TP 3 Exercice 6(cliquer ici pour accéder à Moodle au corrigé du TP 3).

L'idée du tri à bulle est de comparer deux éléments voisins et des les échanger s'ils sont dans le mauvais ordre. On répète ce processus plusieurs fois jusquà ce que la liste soit triée.

tri

```
def tribulle(L : list[int]) -> None :
    for k in range(len(L)-1):
        for pos in range(len(L)-1):
            if L[pos+1]< L[pos] :
                temp=L[pos+1]
                L[pos+1]=L[pos]
                L[pos]=temp
        print(f"  L={L} ")
L = [9,7,3,8,1]
tribulle(L)
```

Votre premier travail consiste à éxécuter *tribulle* tel quel pour voir ce qui se passe. Vous devez repérer la ou les lignes de comparaison et la ou les lignes de permutation. Vous modifierez ce code dans la prochaine question.

Licence Creative Commons

**11)** Adapter le code *tribulle()* suivant en ré-utilisant les fonctions *permute()* et *est_avant()* des questions précédentes.

tri

```
def tribulle(data : list[list[str]]) -> None :
    for k in range(len(data)-1):
        for pos in range(len(data)-1):
            ...
            ...

L4 = [['NOËL', 'NEVEU'], ['MARC', 'MARTIN'],
      ['JULIETTE', 'BOYER'], ['RENÉ', 'SCHNEIDER']]
print(f"Avant tri, liste des Français {L4}")
tribulle(L4)
print(f"Après tri, liste des Français {L4}")
```

La vérification sur la liste L4 doit vous donner le résultat suivant, vous pouvez voir que les noms BOYER, MARTIN , NEVEU et SCHNEIDER sont par ordre alphabétique :

SHELL

```
>>> %Run
[['JULIETTE', 'BOYER'], ['MARC', 'MARTIN'],
['NOËL', 'NEVEU'], ['RENÉ', 'SCHNEIDER']]
```

**12)** Écrire la fonction *trieLesData()* qui reçoit en entrée le dictionnaire dont les valeurs sont les listes des étudiants dans le désordre. Trier chaque liste du dictionnaire à l'aide de la fonction *tribulle()*

t

```
def triLesData(dico : dict)  -> None :
    for pays in dico.keys() :  # parcourt les clés du dictionnaire

triLesData(dico)
print()
for pays in dico.keys() :
    extrait=dico[pays][0:3] # on extrait les 3 premiers étudiants
print(f"{pays} : {extrait}")
```

Licence Creative Commons

SHELL

```
>>> %Run
Pays-Bas : [['TIMO', 'AARDEN'], ['OLAF', 'ADELAAR'], ['FATIMA', 'ADRIAANSEN']]
Allemagne : [['GUENTER', 'ACKERMANN'], ['DANUTA', 'ADLER'], ['KRIEMHILD', 'ADLER']]
Italie : [['UGOLINO', 'ABATE'], ['FIORINO', 'ABATE'], ['STEFANIA', 'ALFONSI']]
Finlande : [['EEMIL', 'AALTO'], ['ANNA', 'AALTONEN'], ['ARI', 'AALTONEN']]
France : [['SABINE', 'ADAM'], ['MARGAUX', 'ALBERT'], ['ARTHUR', 'ALEXANDRE']]
Turquie : [['ALGI', 'AKAR'], ['OLU', 'AKAR'], ['AYAYDIN', 'AKAR']]
Pologne : [['OSKAR', 'AMANOWICZ'], ['ROKSANA', 'ANDRYSZCZYK'], ['FABIAN', 'BAUM']]
```

**13)** Écrire la fonction *sauvegardeLesDonnees()* qui parcourt le dictionnaire afin d'en extraire chaque liste pour la sauvegarder dans un fichier à l'aide de la fonction *ecritFichierCsv()*. Les données des étudiants Français sont à sauvegarder dans le fichier *étudiants_France.csv*, idem pour les autres pays. Les fichiers sont à écrire dans le répertoire *Sortie_csv*.

```
def sauvegardeLesDonnees(dico : dict ) -> None :
    for pays in dico.keys() :
        nomFichier=...
        data = ...
        ecritFichierCsv(nomFichier,data)

sauvegardeLesDonnees(dico )
```

```
.
├── Data_csv
│   └── Liste_1789_etudiants.csv
├── Fonctions_support
│   └── gestionFichier.py
├── main.py
├── Sortie_csv
│   ├── etudiant_Allemagne.csv
│   ├── etudiant_Finlande.csv
│   ├── etudiant_France.csv
│   ├── etudiant_Italie.csv
│   ├── etudiant_Pays-Bas.csv
│   ├── etudiant_Pologne.csv
│   └── etudiant_Turquie.csv
└── tribulle.py

4 directories, 11 files
```

FIGURE 6 – Plan des répertoires avec la commande *tree* dans un terminal.

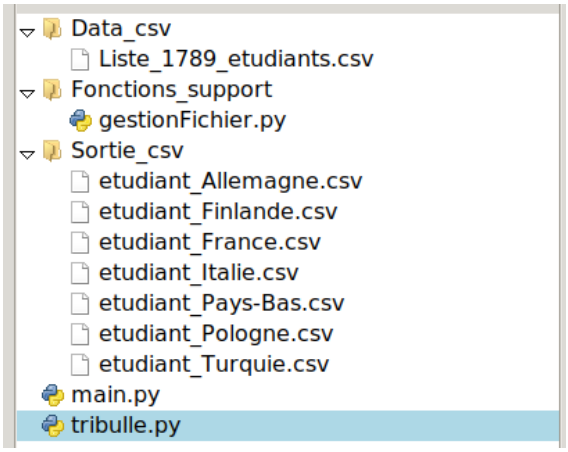


FIGURE 7 – Plan des répertoires vu dans le navigateur Thonny.

**14)** Les fichiers n'ont pas d'entête...allez on corrige !

Licence Creative Commons

**15)** A l'aide de matlplotlib, tracer un histogramme puis un camenbert de la répartition du nombre d'étudiants par pays.

https://matplotlib.org/stable/gallery/lines_bars_and_markers/bar_colors.html

https://matplotlib.org/stable/gallery/pie_and_polar_charts/pie_features.html

**16)** Revenons sur tri, à présent uniquement les noms de famille ont été utilisés. Écrire une fonction trieNomPrénom() qui en cas d'égalité sur les Noms de famille, trie sur les prénoms.

Proposez votre approche. Définissez les cas tests.

Licence Creative Commons